

```
final addTodoMutation = Mutation<void>();  
final addState = ref.watch(addTodoMutation);
```

key difference

- Provider is a tool for representing state
- Mutation is a tool for representing the lifecycle of actions

```
final todosAsync = ref.watch(todoListProvider);
```

```
final addState = ref.watch(addTodoMutation);  
final updateState = ref.watch(updateTodoMutation);  
final deleteState = ref.watch(deleteTodoMutation);
```

Idle → Pending → Success / Error

```
class TodoListState extends Equatable {  
    const TodoListState({  
        this.status = TodoListStatus.loaded,  
        this.todos = const [],  
        this.transientFailure,  
        this.deletingTodoId,  
    });  
  
    final TodoListStatus status;  
    final List<Todo> todos;  
    final String? transientFailure;  
    final String? deletingTodoId;  
  
    ...  
}
```

```
todoListProvider: AsyncValue<List<Todo>>  
  
final addTodoMutation = Mutation<void>();  
final updateTodoMutation = Mutation<void>();  
final deleteTodoMutation = Mutation<void>();
```

- Provider → Data State
- Mutation → Action lifecycle

- todoListProvider → Todo Data State
- addTodoMutation → State of the action to add a Todo
- updateTodoMutation → State of the action to update a Todo
- deleteTodoMutation → State of the action to delete a Todo

`ref.watch(addTodoMutation)`

- subscribing to the action lifecycle
- observes the state of the currently running action

- Provider → What is the current state?
- Mutation → What action is currently in progress?

- **todo_mutation**
 - AsyncNotifer (ViewModel) + Mutation
- **login scenario (action only)**
 - Mutation